



Claude

How can I help you today?



Haiku 4.5 ▾



Write



Learn



Code



Opus 4.7

Most capable for ambitious work

Sonnet 4.6

Most efficient for everyday tasks

Haiku 4.5

Fastest for quick answers



Extended thinking

Think longer for complex tasks



More models



How to Choose the Right Claude Model ?



Claude

How can I help you today?



Haiku 4.5 ▾



Write



Learn



Code



Opus 4.7

Most capable for ambitious work

Sonnet 4.6

Most efficient for everyday tasks

Haiku 4.5

Fastest for quick answers



Extended thinking

Think longer for complex tasks



More models



How to Choose the Right Claude Model ?

What will this video give you?

For Researchers

- Know each model's limits, backed by official system cards
- Balance cost, quality, and latency

For Developers

- Choose the right model for your use case before writing code
- Match inference techniques to model support — not every model supports every technique

For General Users

- Pick the right Claude model with confidence using just 4 questions
- See the clear trade-offs between latency, quality, and cost

Scope

- **Models covered:** the three latest Claude models available as of April 2026 — Opus 4.7, Sonnet 4.6, and Haiku 4.5
- **Inference techniques covered:** streaming, caching, batching, and thinking modes (extended, adaptive)

Ask yourself the following questions:

1. How complex is my task?
2. How long am I willing to wait for a response?
3. What is my budget?
4. What inference techniques do I want to use?

Ask yourself the following question ...?

1. How complex is my task?

Complexity	Model	Examples of Tasks	Anthropic Description
Simple, repetitive, high-volume	Haiku 4.5	Grammar correction, Q&A chatbot, text classification, basic translation, data lookup, email auto-reply	"Fastest model with near-frontier intelligence" — designed for speed and high-volume tasks
Moderate — coding, analysis, reasoning	Sonnet 4.6	Code generation, document summarization, business reporting, math problem solving, financial analysis, web research	"Best combination of speed and intelligence" — sized for moderate-complexity professional tasks
Highest — advanced professional & engineering tasks	Opus 4.7	Complex code debugging, multi-step research, advanced software engineering, real-world professional consulting, scientific reasoning, long-context analysis	"Most capable model for complex reasoning and agentic coding" — designed for the highest-complexity tasks

Ask yourself the following question ...?

2. How long am I willing to wait for a response?

Tolerance	Model	Anthropic's Description
Seconds — need it fast	Haiku 4.5	"The fastest model" — Comparative latency rated Fastest
Minutes — balanced	Sonnet 4.6	Comparative latency rated Fast ; adaptive thinking lets the model regulate reasoning depth automatically
Longer — depth over speed	Opus 4.7	Comparative latency rated Moderate ; uses adaptive thinking with effort levels (low / medium / high / xhigh / max) for fine-grained control over reasoning depth

Ask yourself the following question ...?

3. What is my budget?

Budget	Model	Rationale	Official API Pricing (per 1M tokens)	Token Cost Concern
Low — high-volume, cost-sensitive	Haiku 4.5	Lowest per-token cost; well-suited for high-volume parallel tasks	Input: \$1.00 / Output: \$5.00 — Batch: \$0.50/\$2.50	
Mid — balance of quality and cost	Sonnet 4.6	Best combination of speed and intelligence" at mid-tier pricing	Input: \$3.00 / Output: \$15.00 — Batch: \$1.50/\$7.50	 Thinking tokens (extended or adaptive) are billed at the output rate (\$15/M standard, \$7.50/M batch).  Use adaptive thinking so Claude only thinks when needed — skipping unnecessary reasoning on simple queries.
High — mission-critical output quality	Opus 4.7	Most capable general-access model; justified for complex professional tasks	Input: \$5.00 / Output: \$25.00 — Batch: \$2.50/\$12.50	 New tokenizer in Opus 4.7 may use up to 35% more tokens for the same text compared to previous models

Ask yourself the following question ...?

4. What inference techniques do I want to use? (API-level optimization)

Need	Inference Technique	Model Support	Notes
Real-time output for chatbots/agents	Streaming	All 3 models	Returns tokens as generated — better UX for live apps
Lower cost on repeated context	Prompt Caching	All 3 models	Cache hits cost 10% of input rate — huge savings on long prompts
50% discount, async OK	Batch Processing	All 3 models	Best for bulk jobs; not suitable for real-time apps
Smart auto-control of reasoning depth	Adaptive Thinking	Sonnet 4.6, Opus 4.7	Claude decides per-request; pairs with <code>effort</code> parameter
Predictable fixed thinking budget	Extended Thinking	Haiku 4.5, Sonnet 4.6	You set <code>budget_tokens</code> ; not supported on Opus 4.7

Summary Decision Card

Decision Factor	Haiku 4.5	Sonnet 4.6	Opus 4.7
1. Task Complexity	Simple, repetitive, high-volume (grammar correction, Q&A, classification, translation)	Moderate (code generation, business reporting, document summarization, financial analysis)	Highest (complex debugging, multi-step research, advanced software engineering, professional consulting)
2. Response Wait Time	Seconds — fastest latency	Minutes — adaptive thinking self-regulates depth	Longer — designed for depth over speed
3. Budget (per 1M tokens)	Input: \$1 / Output: \$5 Batch: \$0.50 / \$2.50	Input: \$3 / Output: \$15 Batch: \$1.50 / \$7.50	Input: \$5 / Output: \$25 Batch: \$2.50 / \$12.50 ⚠ New tokenizer may use up to 35% more tokens
4. Inference Techniques	Streaming, Prompt Caching, Batch Processing, Extended Thinking	Streaming, Prompt Caching, Batch Processing, Extended Thinking , Adaptive Thinking	Streaming, Prompt Caching, Batch Processing, Adaptive Thinking (<i>critical to offset tokenizer cost via caching</i>)

Conclusion

Ask yourself these 4 questions:

1. How complex is my task?
2. How long am I willing to wait for a response?
3. What is my budget?
4. What inference techniques do I want to use?

The answer depends on your situation. Pick the model that fits.



Haiku 4.5



Sonnet 4.6



Opus 4.7

Reference

Official Anthropic Documentation

- Anthropic document - <https://platform.claude.com/docs/en/about-claude/models/overview>
- Anthropic API Pricing — <https://platform.claude.com/docs/en/about-claude/pricing>
- Prompt Caching — <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>
- Batch Processing — <https://platform.claude.com/docs/en/build-with-claude/batch-processing>
- Plans & Pricing — <https://claude.com/pricing>

*Content research and writing for this video assisted by Claude (Anthropic)

Happy Learning !